

EC9540 - HUMAN COMPUTER INTERACTION

Lab 04

DE COSTA M.S.M.

2022/E/092

SENARATHNA S.A.D.H.D.

2022/E/141

MediLink: A Community Healthcare Information and Support Platform

Introduction

Healthcare services are essential for improving people's quality of life. However, many patients still face difficulties when accessing hospital services due to manual appointment booking, limited information about doctor and medicine availability and time-consuming payment processes. These challenges often lead to unnecessary hospital visits, long waiting times and inconvenience for patients.

To address these issues, we propose **MediLink**, a web-based healthcare platform designed to improve the patient experience through a simple and user-friendly interface. Based on the findings of our user requirements survey, the scope of the system focuses on three core patient services:

- Checking doctor availability and booking appointments.
- Checking medicine availability before visiting the hospital.
- Making secure online payments while providing patient details.

These features were selected because they directly address the main problems identified through the questionnaire survey. They also provide a complete and convenient user journey, allowing patients to check doctor availability, book appointments, verify medicine availability and complete payments online without unnecessary hospital visits. The system is designed following Human-Computer Interaction (HCI) principles to ensure usability, efficiency and a positive user experience.

Front-End Implementation Overview

MediLink is a community healthcare web platform designed to reduce the inconvenience associated with manual appointment booking, uncertain medicine availability, and time-consuming payment processes. The Lab 03 design established six major patient interfaces, a shallow navigation structure, reusable GUI elements, and an end-to-end booking journey. The current front-end prototype implements this interaction model as a functional browser-based interface.

Reference Design and HCI Objectives

- The design focuses on three core patient services: doctor availability and appointment booking, medicine availability checking and secure online payment.
- The patient interface uses a persistent navigation structure, so users are not trapped in a strict linear wizard.
- Large touch-friendly controls and high-contrast text were specified to support accessibility, including elderly users.
- The design minimizes fields and interaction steps, reflecting Lab 03 finding that easy appointment booking was a major user preference.
- The primary booking path is Landing/Login/Register → Dashboard → Find Doctor → Appointment Slot → Appointment Summary → Patient Details → Payment → Confirmation.

Recommended Professional MERN Front-End Structure

For the frontend implementation stage, the current single-page prototype can be migrated into a maintainable React application without changing the approved user journey. The recommended structure is:

- React + Vite for the front-end application and fast development/build tooling.
- React Router for route-level navigation between authentication, dashboard, doctor search, booking, medicine and appointment screens.
- Tailwind CSS or a consistent component library for maintainable responsive styling. The existing prototype's design tokens can be preserved during migration.
- Lucide React (or an equivalent SVG icon library) for consistent icons.
- React Hook Form + Zod for structured form handling and validation.
- Axios or fetch for HTTPS REST API communication.
- TanStack Query for server-state fetching, caching, loading and error states once the back end is connected.
- Reusable components such as Button, Input, Select, Badge, Card, Alert, Modal, Table and BookingProgress.
- Role-based route protection for patients, doctors and hospital-staff areas when those portals are connected to the back end.

Front-End Architecture

Layer	Responsibility	MediLink Example
Presentation	Renders responsive user interfaces	Dashboard, Find Doctor, Medicine, Payment
Component layer	Reusable visual and interaction elements	Buttons, cards, badges, form fields, chips, tables
State / interaction layer	Maintains temporary UI and booking state	Selected doctor, date, time, payment method
Service/API layer	Communicates with application services	Doctor search, medicine search, appointment and payment APIs
Back-end boundary	Receives and persists business data	Node/Express REST endpoints in the planned MERN system

In the MERN implementation, these responsibilities should be separated into React components, hooks, service modules and REST API calls.

Interface Implementation & Screenshots

The following screenshots are generated from the supplied MediLink prototype. They demonstrate the implemented interface rather than only the Lab 03 wireframes. The implementation retains the major visual and interaction decisions from Lab 03 while applying a more polished design system with teal healthcare branding, cards, badges, responsive navigation, form states and clear primary actions.

Landing / Login / Registration

The landing screen introduces MediLink's core services and provides direct Login and Register actions. The login interface uses a minimal credential form, password visibility control, validation messages and a route to the role-selection portal. The registration screen validates name, email, phone number, password and confirmation fields.

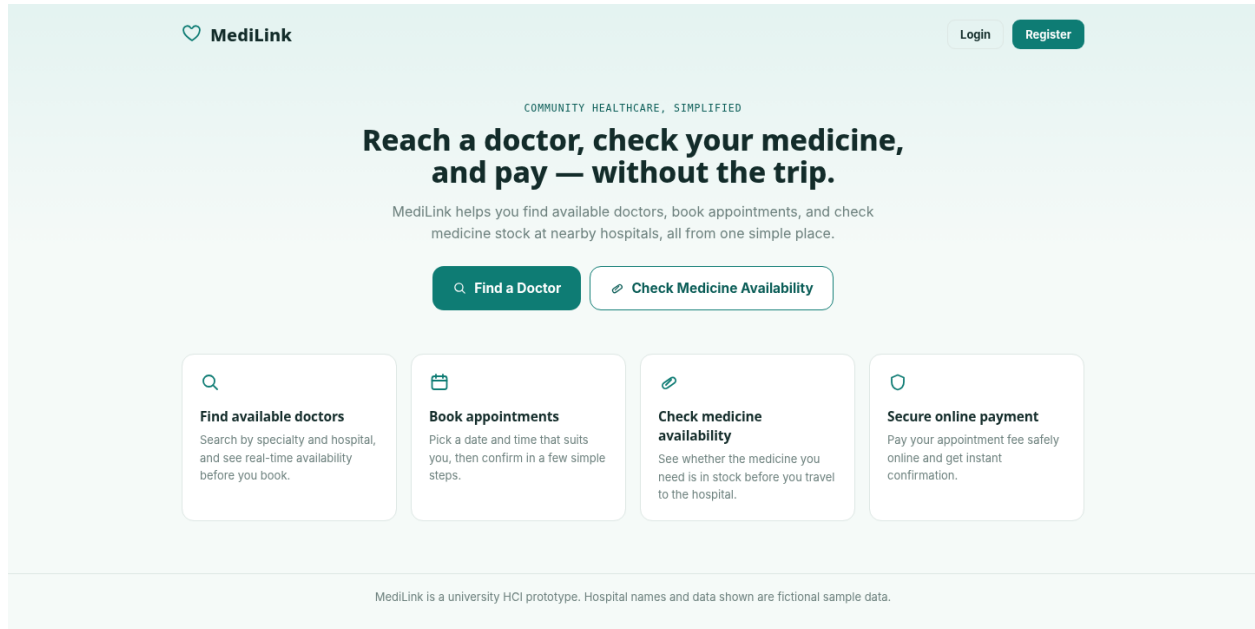


Figure 2.1 - Implemented MediLink landing screen

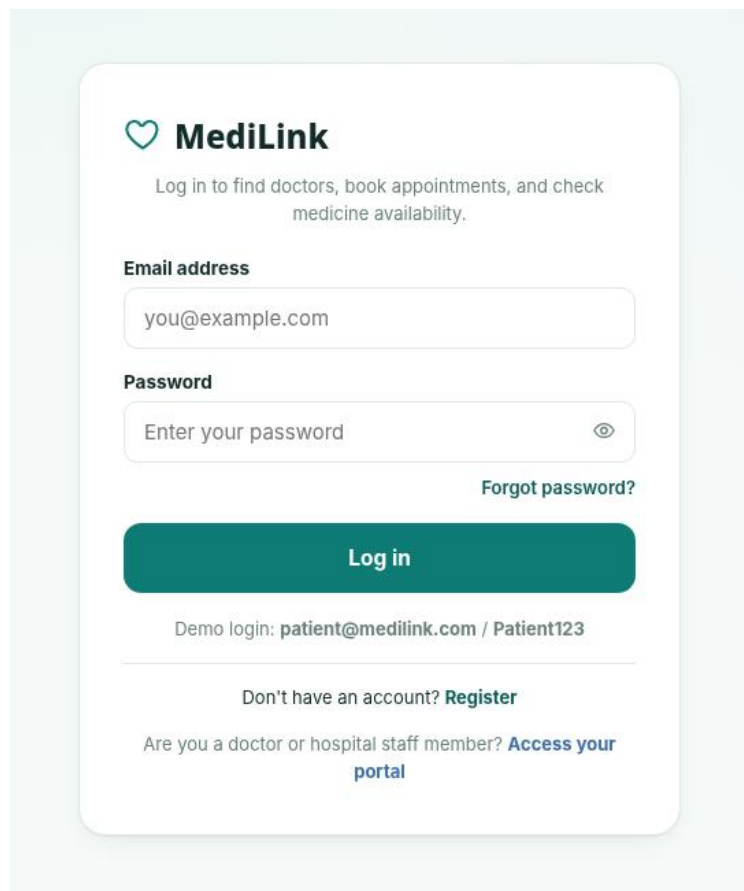


Figure 2.2 - Implemented patient login screen

Patient Dashboard

The dashboard acts as the central patient hub. It presents the four major actions identified in the Lab 03 wireframe: Find a Doctor, Book Appointment, Check Medicine and My Appointments. It also surfaces an upcoming appointment and recent activity, so users do not need to repeatedly navigate for common information.

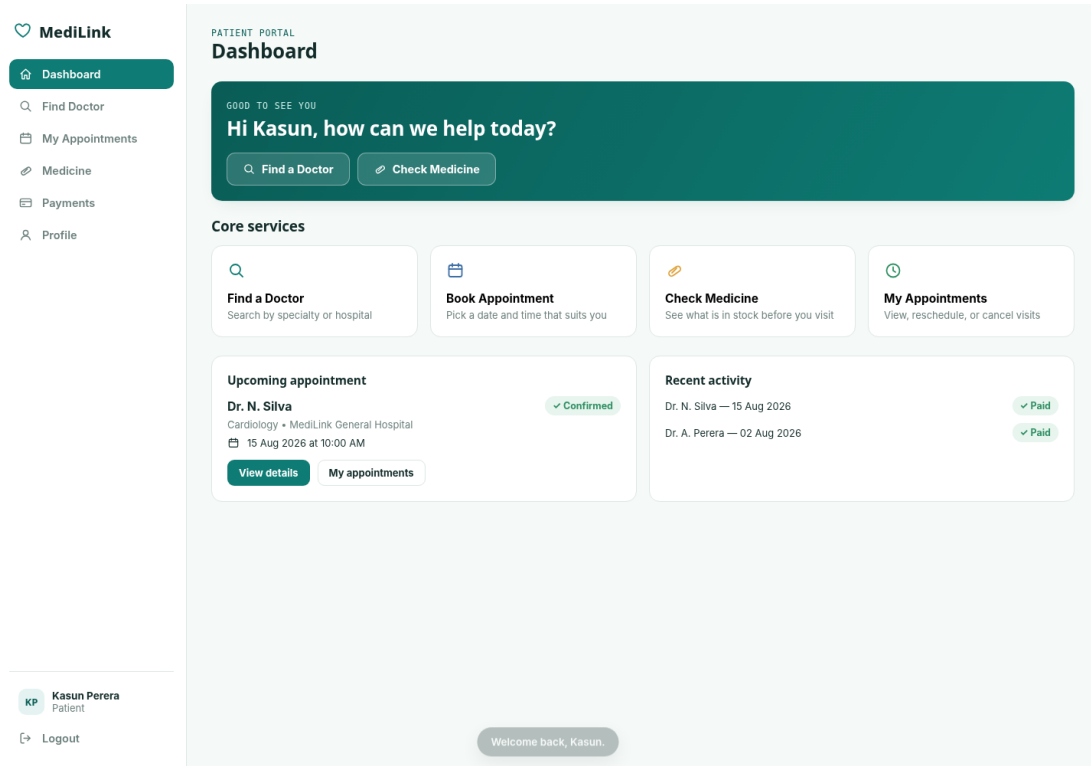


Figure 2.3 - Implemented patient dashboard

Find Doctor and Search Results

The Find Doctor interface implements the planned specialty and hospital filters and an optional doctor-name search. Search results are displayed as doctor cards with specialty, hospital, next available time and an availability badge. The user can view details or proceed directly to booking.

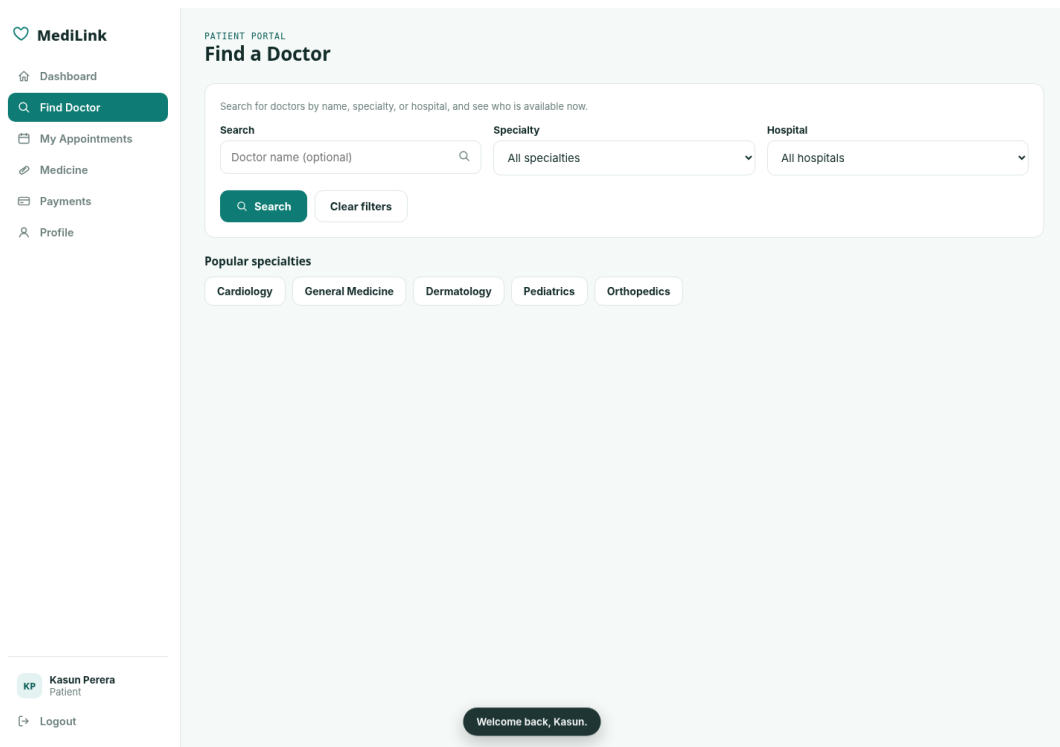


Figure 2.4 - Implemented Find Doctor search interface

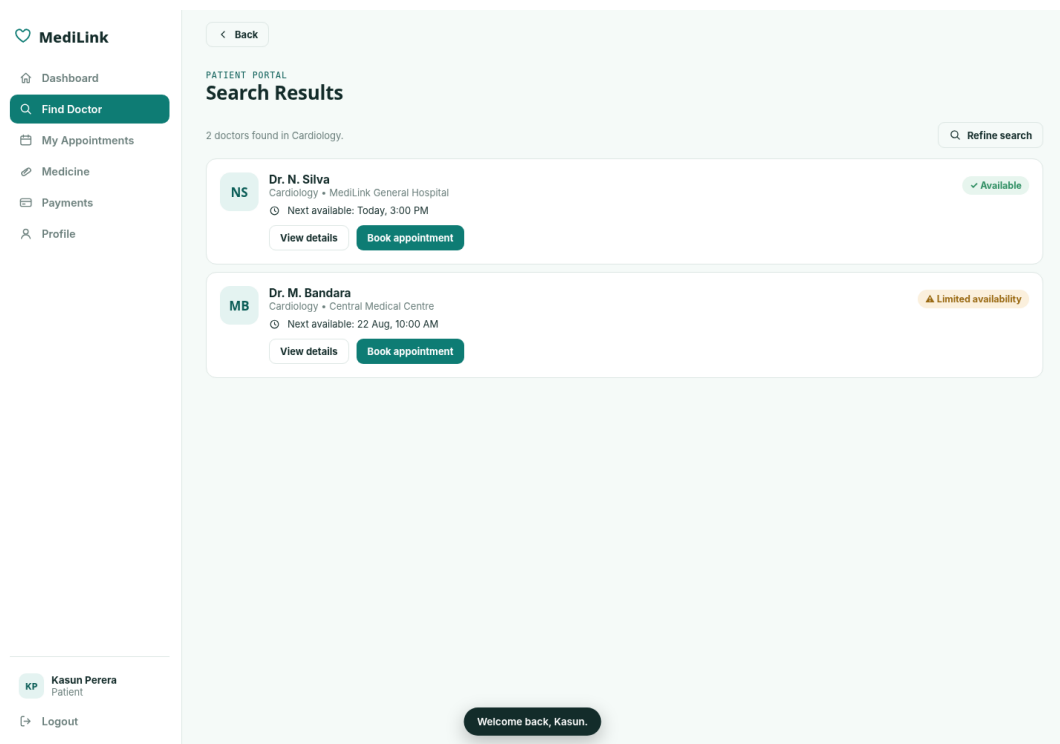


Figure 2.5 - Implemented doctor search results

Doctor Details and Appointment Slot Selection

The doctor details screen provides doctor identity, specialty, hospital, availability, biography and consultation fee. The booking interface then uses date and time chips. The implemented pulse-style progress tracker visually communicates the booking stage from Doctor through Done.

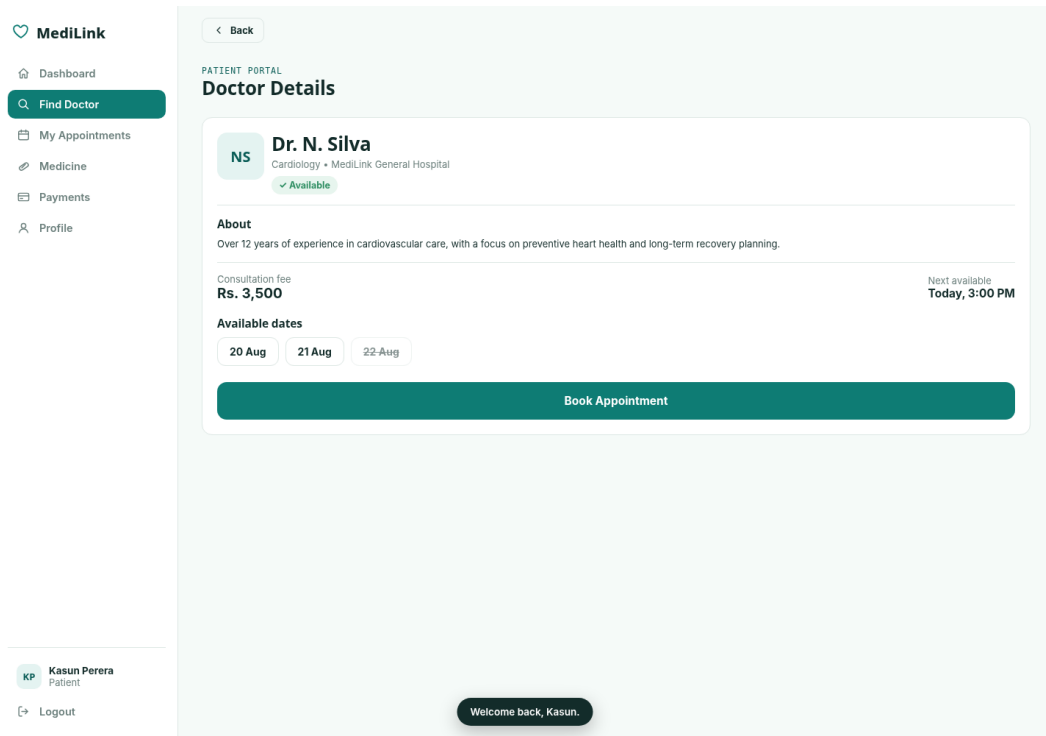


Figure 2.6 - Implemented doctor details screen

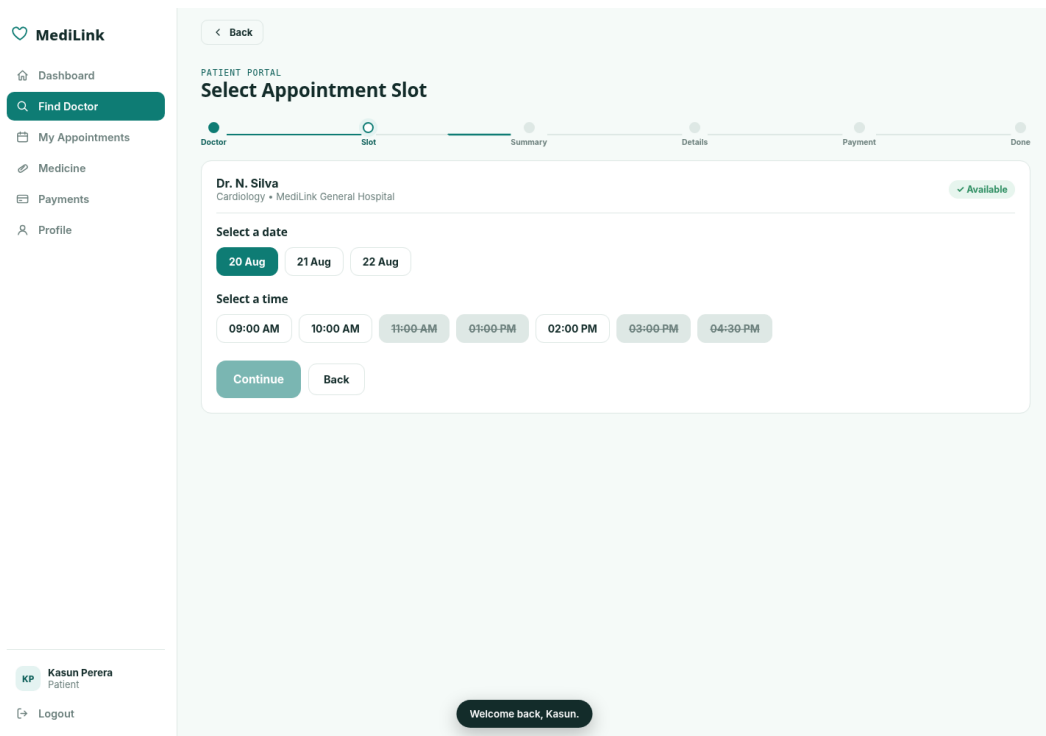


Figure 2.7 - Implemented appointment slot selection

Appointment Summary and Patient Details

Before payment, the prototype presents a confirmation summary containing the selected doctor, specialty, hospital, date, time and consultation fee. Patient details are collected on a separate screen with client-side validation. This follows Lab 03 requirement to confirm important information before committing action.

The screenshot shows the 'Appointment Summary' screen in the MediLink patient portal. The left sidebar contains the MediLink logo, a 'Find Doctor' button, and links to 'My Appointments', 'Medicine', 'Payments', and 'Profile'. The main content area has a 'Back' button and a progress bar with steps: Doctor, Slot, Summary (active), Details, Payment, and Done. The 'Appointment summary' section displays the following information:

Doctor	Dr. N. Silva
Specialty	Cardiology
Hospital	MediLink General Hospital
Date	20 Aug
Time	03:00 PM
Appointment fee	Rs. 3,500

Below the summary is a blue box with a link: 'Need to check if your prescribed medicine is in stock? [Check medicine availability.](#)'. At the bottom are two buttons: 'Confirm Appointment' and 'Back / Edit'.

Figure 2.8 - Implemented appointment summary

The screenshot shows the 'Patient Details' screen in the MediLink patient portal. The left sidebar is identical to the previous screen. The main content area has a 'Back' button and a progress bar with steps: Doctor, Slot, Summary, Details (active), Payment, and Done. The 'Your details' section contains three input fields:

- Full name: Kasun Perera
- Phone number: +94 71 234 5678
- Email address: patient@medilink.com

Below these fields is a 'Continue to Payment' button. To the right, the 'Appointment summary' section displays the same information as in Figure 2.8:

Doctor	Dr. N. Silva
Hospital	MediLink General Hospital
Date	20 Aug
Time	03:00 PM
Fee	Rs. 3,500

Figure 2.9 - Implemented patient details form

Payment Interface

The payment screen implements the planned payment-method selection, appointment summary and security notice. The current prototype supports card and online-banking selections and simulates both successful and failed payment outcomes.

The screenshot displays the 'Payment' screen within the 'MediLink' Patient Portal. The interface is divided into a left sidebar, a main content area, and a bottom status bar.

Left Sidebar:

- MediLink (with heart icon)
- Dashboard
- Find Doctor (highlighted with a magnifying glass icon)
- My Appointments
- Medicine
- Payments
- Profile

Main Content Area:

PATIENT PORTAL
Payment

A progress bar at the top of the main content area shows the following steps: Doctor, Slot, Summary, Details, **Payment** (current step), and Done.

Payment method (with a dropdown arrow icon):

- Card** (selected, with a card icon)
- Online Banking (with a bank icon)

Cardholder name: Name on card

Card number: 1234 5678 9012 3456

Tip: end the number with 0000 to preview a failed payment.

Expiry date: MM/YY

CVV: 123

✔ Your payment information is encrypted and protected. MediLink never stores your full card details.

Buttons: Pay Now (green), Cancel (white)

Order summary (on the right):

Doctor	Dr. N. Silva
Hospital	MediLink General Hospital
Date & time	20 Aug, 03:00 PM
Total	Rs. 3,500

Bottom Status Bar:

KP Kasun Perera Patient
[+] Logout

Figure 2.10 - Implemented payment interface

Medicine Availability and Appointment Management

The medicine workflow supports searching by medicine name and hospital, displaying stock status and the last-updated time. Appointment management includes viewing details, rescheduling and cancellation.

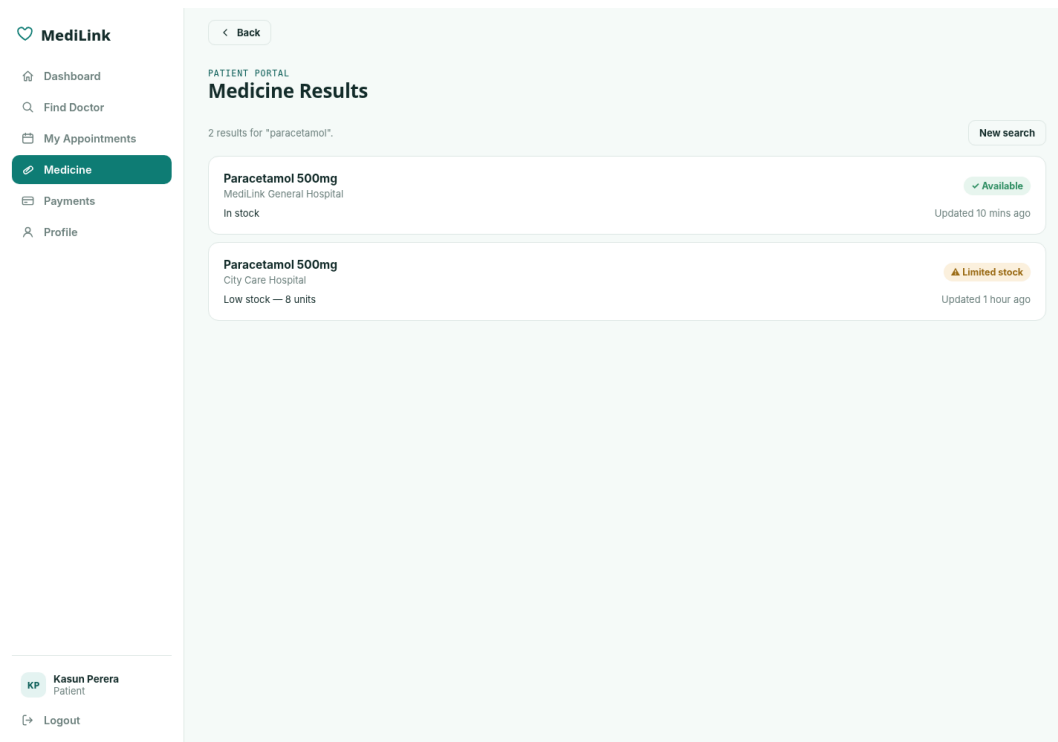


Figure 2.11 - Implemented medicine availability results

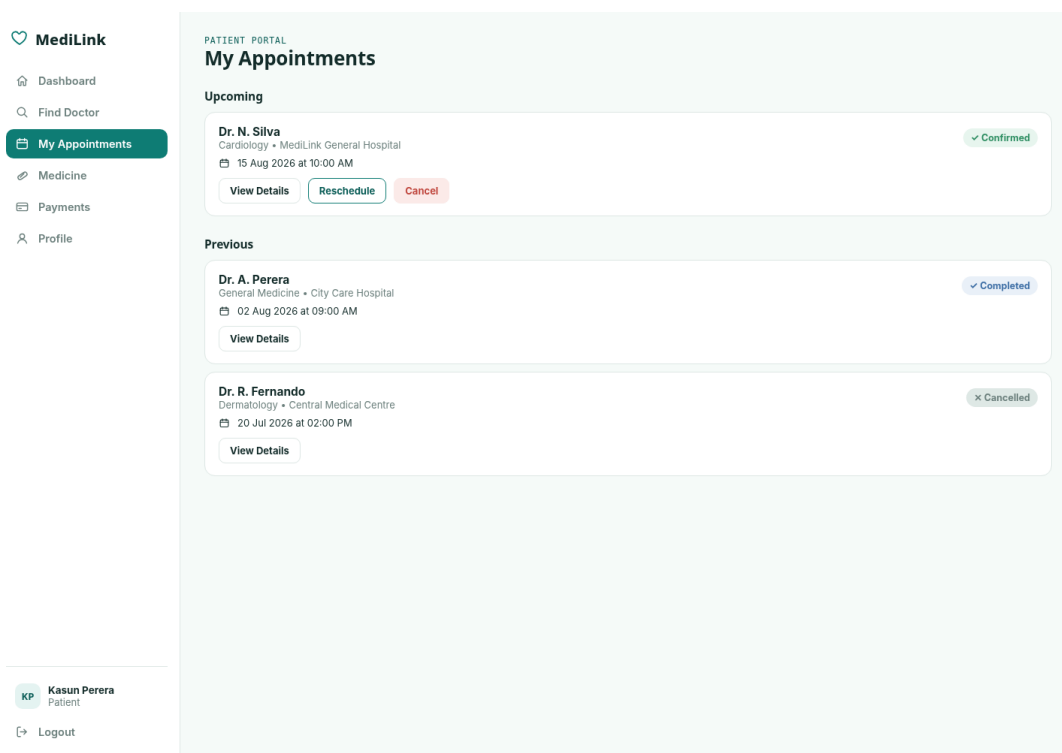


Figure 2.12 - Implemented My Appointments interface

Appointment Details and Confirmation

The appointment-details screen summarizes the hospital, doctor, date, time, appointment status, payment status and amount. The confirmation screen provides a confirmation number, booking information, payment status and navigation back to the appointment record.

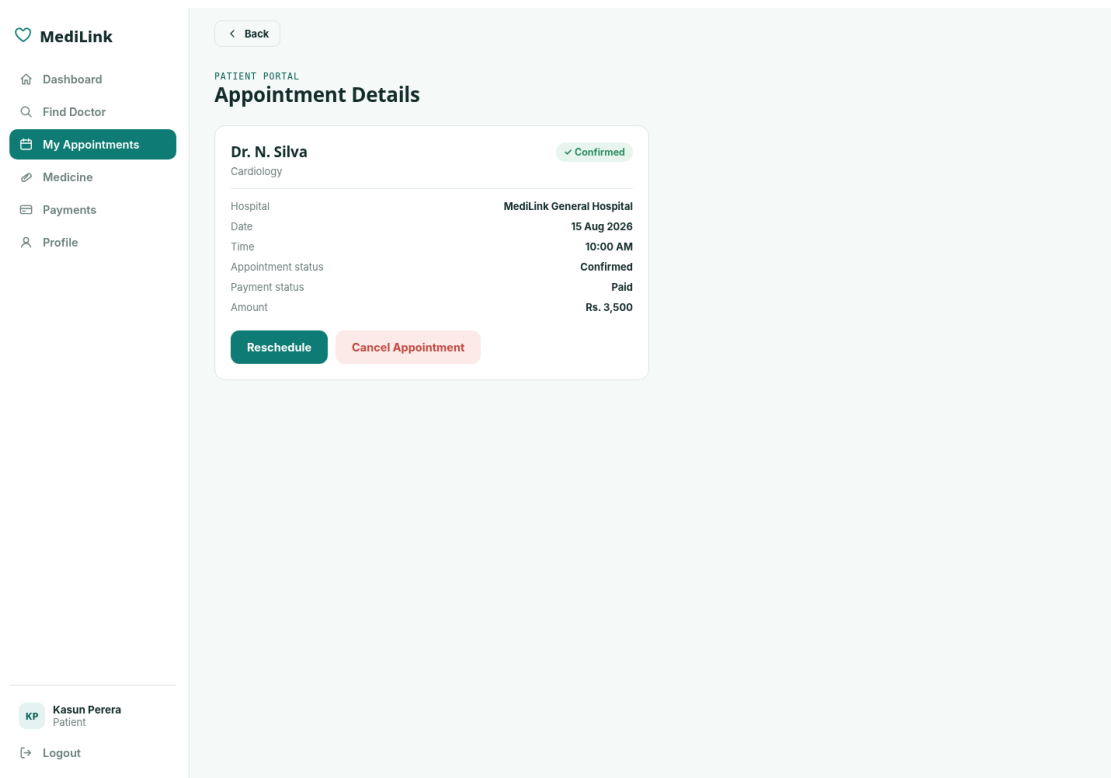


Figure 2.13 - Implemented appointment details

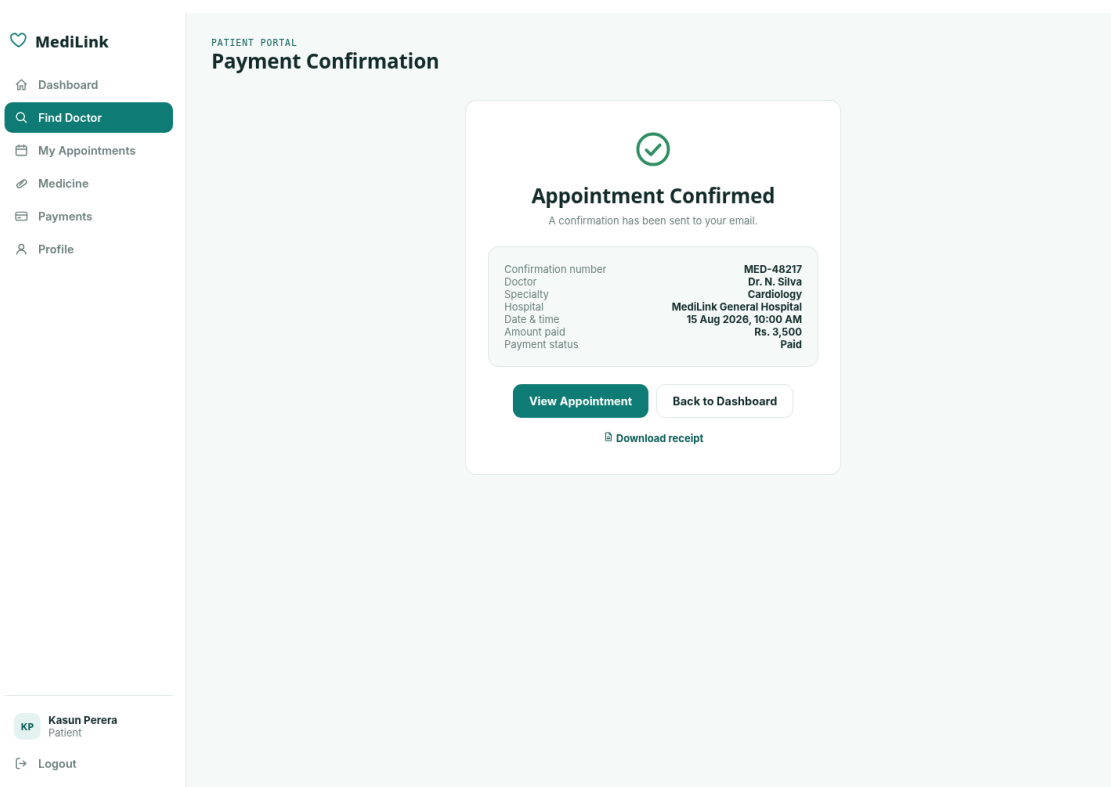


Figure 2.14 - Implemented payment/booking confirmation

Design-to-Implementation Mapping

The following mapping compares the six core interface concepts specified in Lab 03 with the current implemented prototype.

Design Screen	Implemented Prototype	Key Implemented Elements
Login / Register	login / register views	Credential fields, validation, password visibility, role portal link
Dashboard	patientDashboard	Quick-action cards, upcoming appointments, recent activity
Find Doctor & Book Appointment	findDoctor, doctorResults, doctorDetails, selectSlot	Search/filter controls, doctor cards, availability badges, time chips
Medicine Availability	medicineSearch, medicineResults	Medicine search, hospital filter, stock status, update timestamp
Patient Details & Payment	patientDetails, payment	Validated patient form, payment method selection, security notice, fee summary
Booking Confirmation	paymentSuccess / appointmentDetails	Confirmation number, payment status, appointment summary, appointment shortcut

Navigation Implementation

The current JavaScript prototype implements a lightweight client-side navigation model using a central view state and navigation stack. The persistent, authenticated navigation contains Dashboard, Find Doctor, My Appointments, Medicine, Payments and Profile. On smaller screens the sidebar is replaced by a mobile top bar, drawer and bottom navigation.

- **Primary booking route:** Dashboard → Find Doctor → Doctor Details → Select Slot → Appointment Summary → Patient Details → Payment → Confirmation.
- **Medicine shortcut:** Dashboard or booking flow → Medicine Availability → Medicine Results.
- **Appointment record shortcut:** Confirmation → My Appointments / Appointment Details.
- Authenticated users can move to other top-level sections without being trapped in the booking sequence.

GUI Components Implemented

GUI Element	Implementation Evidence	Purpose
Primary / secondary buttons	Reusable btn() helper and button variants	Main and alternative actions
Text fields	Reusable textInput() and field() helpers	Credentials, search terms and patient/payment information
Dropdowns	Native select controls	Specialty, hospital and payment choices
Cards	card, card-flat, doc-card, appt-card	Grouped information and actions
Status badges	badge() / medBadge()	Availability, appointment and payment state

Time chips	chip component	Discrete appointment time selection
Tables	Responsive table wrapper	Payments, doctor schedules and medicine management
Modal	Cancellation confirmation modal	Prevents accidental destructive actions
Toast	Toast notification system	Immediate feedback after actions
Progress tracker	pulseTracker()	Communicates booking progress

Completed Front-End Work

The supplied prototype contains a broad set of complete patient-facing front-end behaviors.

Feature	Completed Front-End Functionality
Authentication UI	Patient login, registration, password visibility, validation feedback and role-portal selection.
Responsive application shell	Desktop sidebar navigation, mobile top navigation, drawer navigation and bottom navigation.
Patient dashboard	Quick service cards, upcoming appointment summary and recent activity.
Doctor discovery	Search by doctor name, specialty and hospital, plus quick specialty selection.
Doctor result presentation	Availability badges, next-available information, doctor details and consultation fee.
Appointment booking	Date selection, time-slot selection, appointment summary and progress tracking.
Patient information	Validated patient name, phone and email fields.
Payment workflow	Card and online-banking options, fee summary, security notice and simulated success/failure paths.
Medicine availability	Medicine search, hospital filtering, stock status and last-updated timestamp.
Appointment management	View appointment details, reschedule appointments and cancel appointments with confirmation.
Profile	Edit basic patient profile details and logout.
Feedback and error handling	Inline validation messages, alert banners, toast notifications, modal confirmation and loading spinner.
Accessibility-oriented styling	Visible keyboard focus, high-contrast controls, touch-friendly sizing and reduced-motion handling.

Validation and Interaction States

- Login validation checks required email/password fields and invalid credentials.
- Registration validates required fields, email format, password length, password confirmation and terms acceptance.
- Patient-detail validation checks name, phone and email before payment.
- Payment validation prevents submission when required payment fields are missing.
- Unavailable doctor and time slots are visually disabled.
- Cancellation uses a confirmation modal to reduce accidental destructive actions.
- Payment processing includes a visible loading state, followed by success or failure feedback.

Responsive Design

Responsive behavior is explicitly implemented through CSS media queries. On desktop-sized screens the application uses a persistent left sidebar. At smaller widths the sidebar is hidden and replaced by a sticky mobile top bar, navigation drawer and bottom navigation. Grid layouts also collapse from four/two columns towards a single-column layout on narrow screens.

Visual Design System

The implementation uses a consistent design-token approach through CSS custom properties. The visual system contains a healthcare-oriented teal primary color, supporting sky/amber/moss/coral states, consistent border radii, shadows, typography, button variants, badges and surface styles. This is a substantial visual refinement over the original basic HTML implementation and provides a good foundation for a React component library.

MERN Migration Plan for the Front End

Current Prototype	React/MERN Equivalent
IEWS router object	React Router route configuration
Global S object	React Context for session/booking state plus local component state
viewPatientDashboard()	PatientDashboard component
viewFindDoctor() / viewDoctorResults()	FindDoctor + DoctorResults components
viewDoctorDetails()	DoctorDetails component
viewSelectSlot()	SlotSelector / BookingStep component
viewPatientDetails()	PatientDetailsForm component
viewPayment()	PaymentForm component
viewMedicineSearch()	MedicineSearch + MedicineResults components
DOCTORS / MEDICINES arrays	REST API responses from Node/Express + MongoDB
toast() / openModal()	Reusable React Toast and Modal components
btn(), field(), badge()	Reusable UI component library

Proposed React Component Structure

```
src/  
  components/  
    ui/          Button, Input, Select, Badge, Card, Alert, Modal, Table  
    booking/     BookingProgress, DoctorCard, SlotSelector, SummaryCard  
    layout/      AppShell, Sidebar, MobileNav, PageHeader  
  pages/  
    auth/        Login, Register, PortalSelect  
    patient/     Dashboard, FindDoctor, DoctorDetails, Booking, Payment  
    medicine/    MedicineSearch, MedicineResults  
    appointments/ MyAppointments, AppointmentDetails, Reschedule  
  services/      authApi, doctorApi, appointmentApi, medicineApi, paymentApi  
  hooks/         useAuth, useDoctors, useAppointments, useMedicine  
  routes/        ProtectedRoute, role-based routes  
  App.jsx  
  main.jsx
```

API Integration Boundary

The architecture specifies that the front end should communicate with the application tier through HTTPS REST endpoints returning JSON. The React implementation should therefore avoid embedding business logic inside page components. Instead, pages should call service functions such as `getDoctors()`, `getDoctorAvailability()`, `bookAppointment()`, `searchMedicine()`, `initiatePayment()` and `getPatientAppointments()`.

Security Considerations

- Do not store plain-text passwords in the front end or database.
- Use HTTPS for production API communication.
- Do not store full card details in MediLink. A PCI-compliant external payment gateway should handle card processing.
- Use authenticated and role-restricted API routes for patient, doctor and staff operations.
- Validate important data on both client and server; client-side validation is for usability, not security.

Conclusion

The front-end implementation of MediLink successfully transforms the interface designs developed in Lab 03 into an interactive healthcare platform prototype. The implemented system covers the main patient-facing functions, including authentication, patient dashboard, doctor search, appointment booking, patient details, payment, booking confirmation, medicine availability, appointment management and profile management. The front end is also structured for future migration to a React-based MERN architecture, allowing the system to relate to Node.js, Express, and MongoDB backend services. Development can focus on implementing real-time data, secure authentication, appointment management, medicine availability updates and integration with a secure external payment gateway.

Overall, the completed implementation provides a strong foundation for developing MediLink into a complete and scalable community healthcare platform while maintaining the usability and interaction principles established during the HCI design process.